Benchmarking de código C++ paralelo generado por inteligencia artificial: convolución 2D de imágenes con OpenMP

Esteban David Muñoz Bautista[*] Isabela Gutierrez Reyes[**]

Resumen

Objetivo: cuantificar el efecto de la paralelización sobre una convolución 2D en C++ y comparar código paralelo generado por inteligencia artificial (IA) con código paralelo escrito por personas. **Materiales y métodos:** se evaluaron cuatro variantes con una interfaz común: secuencial, paralela generada directamente por IA (Claude Opus 5.5), paralela generada por IA con optimización dirigida por el equipo y una implementación OpenMP de un repositorio público de 2016. Se usaron imágenes de 512^2 a 4096^2 píxeles, kernels de 3×3 a 11×11 y 1, 2, 4, 8 y 16 hilos, con 30 repeticiones por configuración y verificación bit a bit contra la versión secuencial. **Resultados y discusión:** en 4096^2 la variante de IA alcanzó speedups de 36,9 a 61,8 frente a la secuencial, salvo con 11×11 (12,7), donde perdió su vectorización; la optimización dirigida lo elevó a 48,0. La implementación humana logró entre 5,8 y 6,7. La escalabilidad relativa a un hilo fue similar en todas (6 a 10 con 16 hilos). **Conclusiones:** el código de IA fue correcto al primer intento y mucho más rápido por vectorización, pero mostró una debilidad dependiente de los datos que solo se detectó midiendo.

Palabras clave: programación paralela; OpenMP; convolución 2D; código generado por IA; benchmarking; speedup.

Abstract

Objective: to quantify the effect of parallelization on a C++ 2D convolution and to compare AI-generated parallel code with human-written parallel code. **Methods and materials:** four variants sharing one interface were evaluated: sequential, parallel generated directly by AI (Claude Opus 5.5), AI-generated with team-directed optimization, and an OpenMP implementation from a 2016 public repository. Images from 512^2 to 4096^2 pixels, 3×3 to 11×11 kernels and 1–16 threads were used, with 30 runs per configuration and bit-exact validation against the sequential version. **Results and discussion:** on 4096^2 images the AI variant reached speedups of 36.9–61.8 over the

[*]Pontificia Universidad Javeriana Cali, Facultad de Ingeniería, Carrera de Ingeniería de Sistemas y Computación. Correo: estebanmunoz13@javerianacali.edu.co

[**]Pontificia Universidad Javeriana Cali, Facultad de Ingeniería, Carrera de Ingeniería de Sistemas y Computación. Correo: isabelagutierrez@javerianacali.edu.co

sequential code, except for 11×11 (12.7), where it lost vectorization; the directed optimization raised it to 48.0. The human implementation reached 5.8–6.7. Scalability relative to one thread was similar for all variants (6–10 at 16 threads). **Conclusions:** the AI code was correct on the first attempt and much faster due to vectorization, but showed a data-dependent weakness that only measurement revealed.

Keywords: parallel programming; OpenMP; 2D convolution; AI-generated code; benchmarking; speedup.

1. Introducción

Los asistentes de programación basados en modelos de lenguaje generan hoy código funcional en lenguajes de alto rendimiento [1]. Estudios recientes han evaluado su capacidad para producir núcleos paralelos con OpenMP, CUDA y otros modelos de programación, principalmente en términos de corrección y, en menor medida, de rendimiento [2,3]. Para un ingeniero, la pregunta práctica no es solo si el código compila y es correcto, sino cuánto rendimiento entrega frente a una implementación escrita por personas y frente a la versión secuencial.

Este trabajo estudia esa pregunta con un núcleo clásico de procesamiento de imágenes: la convolución 2D [4]. El objetivo general es determinar cuantitativamente el impacto de la paralelización sobre el rendimiento de código C++ generado por IA, bajo condiciones idénticas de hardware, software, imágenes de entrada y parámetros de convolución. Los objetivos específicos son: (i) medir tiempo de ejecución, speedup, eficiencia y desviación estándar de cuatro variantes; (ii) analizar su escalabilidad con 1 a 16 hilos; (iii) comparar el código generado directamente por IA, el código de IA optimizado bajo dirección del equipo y una implementación paralela escrita por personas; y (iv) verificar la equivalencia funcional estricta de todas las variantes.

2. Materiales y métodos

2.1. Entorno de hardware y software

Todas las mediciones se realizaron en una sola máquina, descrita en la Tabla 1. Los datos se registraron automáticamente con un script incluido en el repositorio. No se fijó afinidad de hilos ni política de espera de OpenMP (variables `OMP_PROC_BIND` y `OMP_WAIT_POLICY` sin definir), de modo que se midió el comportamiento por defecto del runtime.

Tabla 1: Entorno experimental.

Componente	Valor
CPU	Intel Core i7-10700F, 2,90 GHz, 8 núcleos físicos, 16 hilos lógicos (HyperThreading)
Caché	L2 2 MiB, L3 16 MiB
Memoria	32 GB
Sistema operativo	Windows 10 Pro 10.0.19045
Compilador	GCC 16.2.0 (WinLibs, MinGW-w64 x86_64, UCRT, hilos POSIX) [5]
Flags	`-O2 -fopenmp -std=c++17 -static` (sin `-march=native` ni `-ffast-math`)
Plan de energía	Perfil personalizado de alto rendimiento del usuario (“Plus-Ultra”)
Análisis	Python 3.12.6, NumPy 2.1.3, Matplotlib 3.9.2

2.2. Problema y contrato funcional

Sea I una imagen en escala de grises de $w \times h$ píxeles de 8 bits y K un kernel cuadrado de $k \times k$ pesos enteros no negativos, con $r = \lfloor k/2 \rfloor$. Para cada píxel se calcula

$$A(x, y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} I(y + i - r,\ x + j - r)\, K(i, j), \tag{1}$$

con $I = 0$ fuera de la imagen (zero-padding), acumulando en enteros de 32 bits. La salida es

$$O(x, y) = \text{mín}\left(255,\ \text{máx}\left(0,\ \left\lfloor \frac{A(x, y) + \lfloor S/2 \rfloor}{S} \right\rfloor\right)\right), \qquad S = \sum_{i,j} K(i, j). \tag{2}$$

Se usaron kernels binomiales (aproximación entera de un filtro gaussiano), construidos como el producto exterior de la fila k del triángulo de Pascal. Para $k = 11$ la suma es $S = 2^{20}$ y el peso máximo es $252^2 = 63\,504$; el acumulador máximo, $255 \cdot 2^{20} \approx 2{,}7 \times 10^8$, cabe en un entero de 32 bits. Como la suma de enteros es asociativa y conmutativa, el resultado no depende del orden de acumulación; esto permite exigir igualdad bit a bit entre variantes, a diferencia de una implementación en punto flotante.

Las imágenes se generan en memoria con un generador pseudoaleatorio xorshift de 32 bits con semilla fija ($42 + n$ para una imagen de $n \times n$). El costo de la convolución no depende del contenido de la imagen, por lo que las imágenes sintéticas no sesgan el tiempo medido y garantizan reproducibilidad.

2.3. Variantes evaluadas

Las cuatro variantes implementan la misma firma `convolve(in, out, w, h, k, ksize, nthreads)`.

Secuencial. Cuatro bucles anidados que aplican directamente (1) y (2) con verificación de límites; es la referencia de corrección y de tiempo.

IA directa. Se entregó a Claude Opus 5.5 (Anthropic), en una sesión nueva y sin acceso al repositorio, un prompt fijo que describe la firma, la semántica exacta, los flags de compilación, los tamaños y los conteos de hilos, y pide optimizar para CPU x86-64 multinúcleo. El código devuelto se usó sin modificaciones. El prompt y la respuesta completos están en el directorio `ai_log/` del repositorio. La IA produjo, al primer intento, una versión con reparto estático de filas por hilo, un búfer circular de filas convertidas a enteros de 16 bits y vectorización manual con instrucciones SSE2 (`_mm_madd_epi16`) [6]. Esta vectorización solo se activa si todos los pesos caben en un entero de 16 bits con signo ($\leq$ 32 767); en caso contrario usa un camino escalar.

IA + optimización dirigida por el equipo. A partir del análisis del código anterior y de una medición exploratoria, se propusieron cuatro optimizaciones: (O1) mantener la ruta SIMD con pesos de hasta 16 bits sin signo descomponiendo cada peso como $256\,h + l$ con $h, l \leq 255$, y combinar $A = 256\,A_h + A_l$; (O2) convolución separable; (O3) AVX2 con despacho en tiempo de ejecución; (O4) una limpieza que elimina una advertencia del compilador. El equipo aprobó O1 y O4 y rechazó O2, porque cambia el algoritmo (de k^2 a $2k$ productos por píxel) y rompe la comparabilidad, y O3, porque usaría un conjunto de instrucciones no disponible para las demás variantes con los mismos flags. La implementación de los cambios aprobados se hizo con asistencia de IA; las decisiones están registradas en el repositorio.

Manual (GitHub). Se tomó el método `Convolutionmp::Compute` del repositorio público `rdb987/2DConvolution` (commit `d5b3922`, marzo de 2016) [7]. Se eligió por ser anterior a la disponibilidad de asistentes de código basados en modelos de lenguaje, lo que hace verosímil su autoría humana, y por usar aritmética entera, kernels de tamaño arbitrario y solo OpenMP. Se adaptó lo mínimo: firma común, zero-padding en lugar de bordes periódicos, normalización (2) y `num_threads(nthreads)`. Se conservaron su bucle aplanado sobre todos los píxeles (con división y módulo para obtener fila y columna) y su `#pragma omp for` con planificación por defecto. Las adaptaciones están documentadas en un diff en el repositorio. Esta implementación es una paralelización directa y no aplica optimizaciones de bajo nivel.

2.4. Protocolo de medición

Se evaluaron 4 tamaños (512^2, 1024^2, 2048^2, 4096^2), 4 kernels (3×3, 5×5, 7×7, 11×11) y 5 conteos de hilos ($p \in \{1, 2, 4, 8, 16\}$). La variante secuencial se midió solo con $p = 1$. En total hubo 256 configuraciones y 7 680 mediciones. Para cada configuración y variante:

1. Se ejecutó una vez sobre un búfer de salida previamente llenado con el valor `0xAB` y se comparó byte a byte con la salida secuencial; cualquier diferencia aborta el experimento.
2. Se ejecutó una vez más como calentamiento, descartando el tiempo.
3. Se realizaron 30 ejecuciones medidas con `std::chrono::steady_clock`, tomando el tiempo exclusivamente alrededor de la llamada a `convolve` (sin generación de datos, E/S ni validación).

El orden de las variantes se rotó en cada configuración para repartir efectos de calentamiento térmico y de caché. Antes del experimento se ejecutó una batería de pruebas automáticas de equivalencia: 5 formas de imagen (incluidas 1×1, 5×3 —más pequeña que el kernel— y formas no cuadradas), los 4 kernels y 6 conteos de hilos (incluido $p = 16$ con solo 3 filas),

además de un kernel identidad 1×1 y una imagen saturada. Todas las variantes superaron la batería y las 256 validaciones del benchmark.

2.5. Métricas

Para cada configuración con $n = 30$ tiempos $t_1, \ldots, t_n$ se calculan la media, la desviación estándar muestral y el coeficiente de variación:

$$\bar{T} = \frac{1}{n}\sum_{i=1}^{n} t_i, \qquad \sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(t_i - \bar{T})^2}, \qquad CV = \frac{\sigma}{\bar{T}}. \tag{3}$$

El speedup y la eficiencia se calculan como

$$S_p = \frac{T_{seq}}{T_p}, \qquad E_p = \frac{S_p}{p}, \tag{4}$$

donde T_{seq} es el tiempo medio de la variante secuencial (el T_1 de la definición clásica) y T_p el tiempo medio de la variante evaluada con p hilos. Como una variante paralela puede ser más rápida que la secuencial incluso con un hilo (por ejemplo, por vectorización), también se reporta el speedup relativo $S_p^{rel} = T_1^{(v)}/T_p^{(v)}$, que usa el tiempo de la misma variante con un hilo y aísla la escalabilidad por hilos [8, 9, 10].

3. Resultados

La Tabla 2 resume, para cada imagen y kernel, el tiempo secuencial y el mayor speedup S_p alcanzado por cada variante paralela, con el número de hilos en que se obtuvo. La tabla completa de 4096^2 (tiempo medio $\pm\sigma$, S_p y E_p para cada p) está en el Anexo A; las de los demás tamaños y el CSV con las 7 680 mediciones están en el repositorio.

Tabla 2: Tiempo secuencial y mayor speedup S_p por variante (entre paréntesis, hilos en que se obtuvo).

Imagen	Kernel	T_{seq} (ms)	IA directa	IA + opt. equipo	Manual (GitHub)
512×512	3×3	2.50	19.56 (16)	19.64 (16)	5.00 (16)
512×512	5×5	5.35	34.66 (16)	35.52 (8)	6.24 (16)
512×512	7×7	9.36	35.30 (16)	45.91 (8)	4.82 (8)
512×512	11×11	21.99	13.20 (8)	47.02 (8)	6.04 (8)
1024×1024	3×3	10.10	22.88 (16)	23.23 (16)	4.60 (16)
1024×1024	5×5	21.53	50.63 (16)	50.67 (16)	6.52 (16)
1024×1024	7×7	37.34	64.84 (16)	54.87 (16)	6.62 (16)
1024×1024	11×11	87.94	11.75 (8)	54.68 (16)	6.27 (16)
2048×2048	3×3	39.92	37.14 (16)	37.72 (16)	5.98 (16)
2048×2048	5×5	86.83	52.15 (16)	54.30 (16)	6.69 (16)
2048×2048	7×7	150.55	67.68 (16)	66.13 (16)	6.88 (16)
2048×2048	11×11	351.98	15.04 (16)	57.44 (16)	6.90 (16)
4096×4096	3×3	162.56	36.89 (16)	36.66 (16)	6.17 (16)
4096×4096	5×5	344.52	56.01 (16)	57.06 (16)	6.73 (16)
4096×4096	7×7	597.69	61.76 (8)	58.37 (8)	6.74 (16)
4096×4096	11×11	1429.85	12.70 (16)	47.95 (16)	5.79 (16)

En 4096^2, la variante secuencial tardó 162,6, 344,5, 597,7 y 1 429,8 ms con kernels de 3, 5, 7 y 11, respectivamente. Con un solo hilo, la variante de IA directa ya fue 4,7, 7,2 y 8,9 veces más rápida que la secuencial con kernels de 3, 5 y 7, pero solo 2,0 veces con 11×11. Con 16 hilos alcanzó $S_{16} = 36{,}9$, 56,0, 55,5 y 12,7. La variante con optimización dirigida se comportó igual que la de IA directa en 3, 5 y 7 (diferencias menores que la variabilidad medida), y en 11×11 redujo el tiempo de un hilo de 710,7 a 182,9 ms y el de 16 hilos de 112,5 a 29,8 ms ($S_{16} = 48{,}0$). La implementación humana fue más lenta que la secuencial con un hilo (S_1 entre 0,64 y 0,81) y alcanzó S_{16} entre 5,8 y 6,7. La Figura 1 muestra S_p y la Figura 2, los tiempos con su desviación estándar.

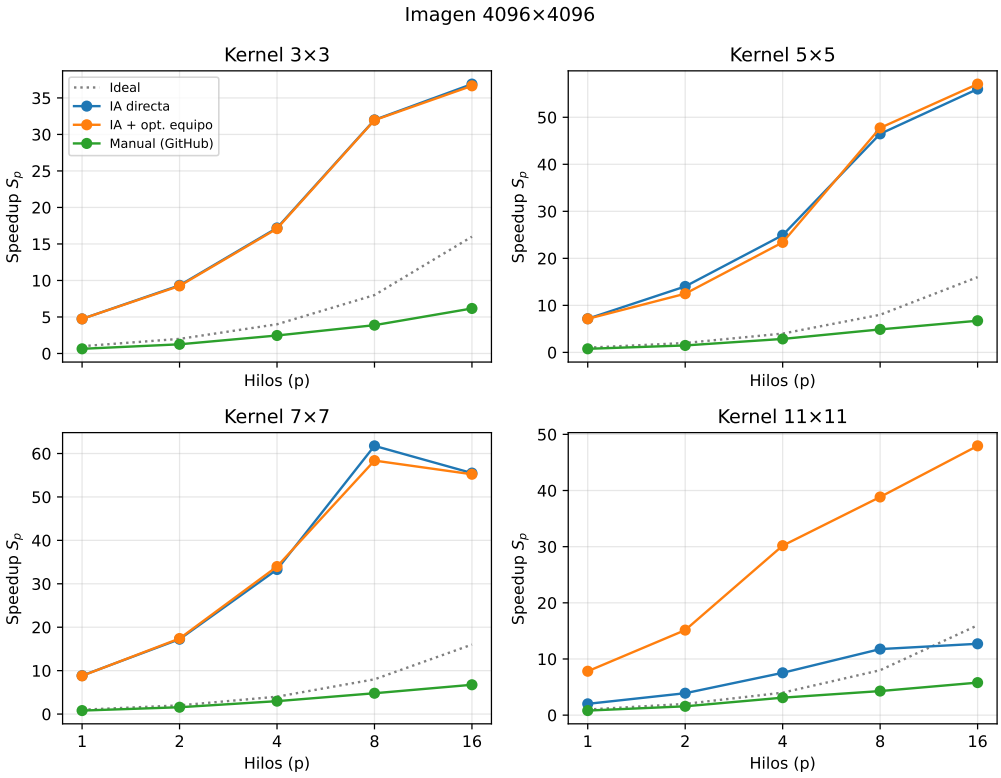


Figura 1: Speedup $S_p = T_{seq}/T_p$ en imágenes de 4096^2. La línea punteada es el speedup ideal $S_p = p$.

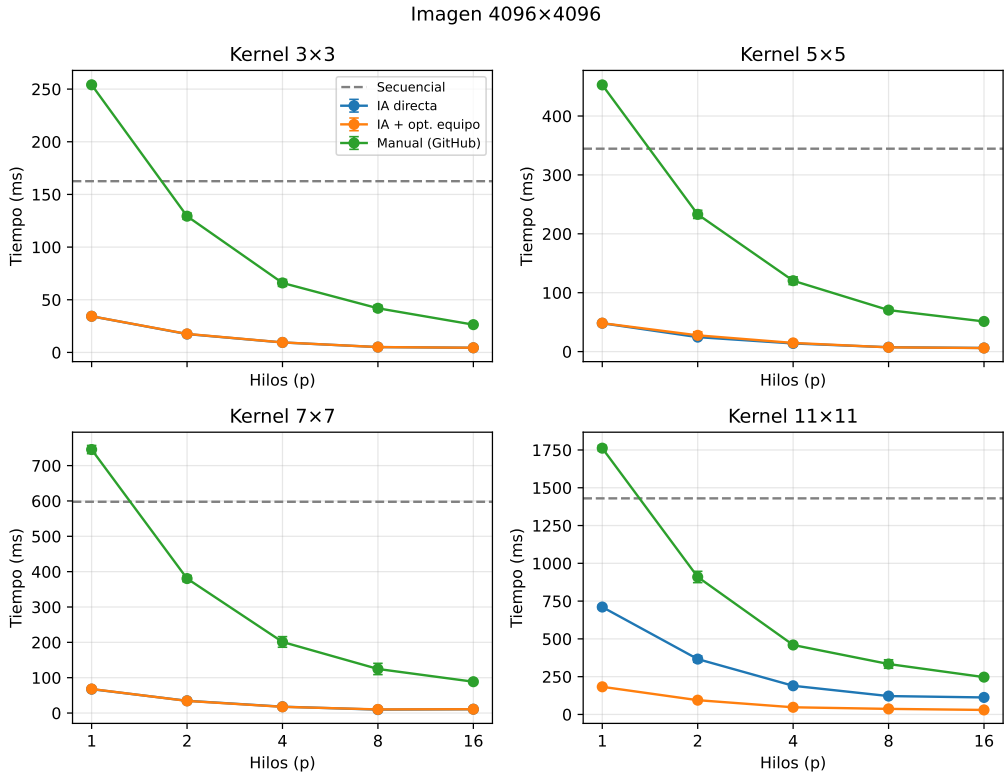


Figura 2: Tiempo medio con barras de $\pm\sigma$ en imágenes de 4096^2. La línea discontinua es el tiempo secuencial.

La Figura 3 muestra la escalabilidad respecto a cada variante con un hilo. Hasta 4 hilos las tres variantes están cerca del ideal (S_4^{rel} entre 3,3 y 3,9). Con 8 hilos se ubicaron entre 5,0 y 7,0, y con 16 entre 6,1 y 9,6. La variante humana obtuvo el mayor speedup relativo con 16 hilos en los cuatro kernels (7,1 a 9,6 frente a 6,1 a 8,0 de las variantes de IA). Con 7×7 en 4096^2, las variantes de IA fueron más lentas con 16 hilos que con 8 (10,8 frente a 9,7 ms en la variante de IA directa).

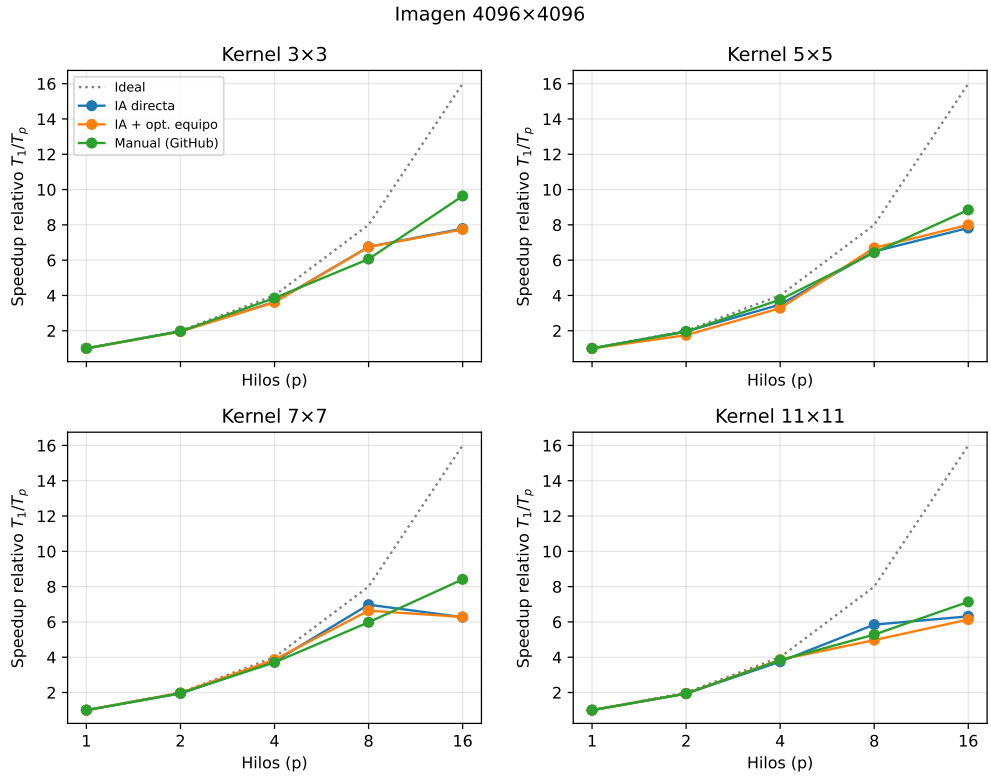


Figura 3: Speedup relativo $T_1^{(v)}/T_p^{(v)}$ de cada variante en imágenes de 4096^2.

La eficiencia respecto a la secuencial (Anexo A) es mayor que 1 en las variantes de IA, porque su ventaja con un hilo es mayor que 1, y disminuye con p: en 4096^2 con 11×11, $E_{16} = 0{,}79$ (IA directa), 3,00 (IA + optimización) y 0,36 (manual).

En cuanto a la variabilidad, el *CV* medio fue 1,2 % con 1 hilo, 5,0 % con 2, 9,1 % con 4, 16,5 % con 8 y 9,4 % con 16. En 78 de las 256 configuraciones el *CV* superó el 10 %; el grupo más numeroso (36) corresponde a 8 hilos, y 59 de ellas tienen tiempos medios menores que 10 ms. En la variante secuencial el *CV* no superó el 6,2 %.

4. Discusión

IA frente a código humano. La mayor diferencia entre variantes no proviene del paralelismo sino de la eficiencia por núcleo. La IA aplicó, sin que se le pidiera explícitamente, técnicas de bajo nivel (vectorización SSE2, búfer circular de filas, normalización con multiplicación) que dan una ventaja de 4,7 a 8,9 veces con un solo hilo. La implementación humana, en cambio, es una paralelización directa: su bucle aplanado calcula una división y un módulo por píxel y verifica límites en el bucle más interno, lo que explica que sea más lenta que la secuencial con un hilo. Por eso, en S_{16} frente a la secuencial, el código de IA directa es entre 6,0 y 8,3 veces superior al humano con kernels de 3 a 7 (2,2 veces con 11×11), mientras que en escalabilidad relativa la implementación humana es ligeramente mejor. Es decir, el código humano usado aquí representa una implementación paralela típica, no una implementación manual optimizada; con este experimento no se puede afirmar que la IA supere a un experto que optimice a mano.

Debilidad dependiente de los datos. El código de IA cumplió la especificación en todos los

casos, pero su ruta rápida dependía de que los pesos cupieran en 16 bits con signo. Con el kernel binomial 11×11 (pesos de hasta 63 504) cayó a un camino escalar y su speedup bajó de 55,5 (7×7) a 12,7. Esta limitación no aparece en el prompt ni en las pruebas de corrección; solo se detectó al medir. La optimización O1, aprobada por el equipo tras leer el código, recuperó la vectorización sin cambiar el algoritmo ni perder exactitud, y fue la única que cambió el rendimiento de forma apreciable.

Efecto de HyperThreading y de la memoria. El procesador tiene 8 núcleos físicos; con $p = 16$ dos hilos comparten cada núcleo. Las variantes de IA ya usan intensamente las unidades vectoriales de cada núcleo, por lo que el segundo hilo lógico aporta poco (S^{rel} pasa de 6,7 a 7,8 en 3×3) o incluso empeora el tiempo (7×7). La variante humana, escalar y con más instrucciones por píxel, deja recursos libres que el segundo hilo aprovecha (de 6,1 a 9,6 en 3×3). Esta explicación es una hipótesis coherente con los datos; no se realizaron mediciones con contadores de hardware para confirmarla. El kernel de 3×3 tiene la menor intensidad aritmética, por lo que es el caso más expuesto al ancho de banda de memoria.

Variabilidad. La variabilidad crece con el número de hilos, especialmente con 8, cuando todos los núcleos físicos están ocupados y cualquier proceso del sistema operativo compite con un hilo de cómputo. Los tiempos con CV alto corresponden mayoritariamente a ejecuciones cortas, donde una interrupción pesa más. Por ello, las diferencias de pocos puntos porcentuales entre la variante de IA directa y la optimizada en kernels de 3, 5 y 7 no deben interpretarse como diferencias reales.

4.1. Amenazas a la validez

- **Una sola máquina y un solo sistema operativo:** los resultados dependen de la microarquitectura y del planificador de Windows; no se fijó afinidad de hilos.
- **Una sola generación de IA:** los modelos de lenguaje no son deterministas; otra ejecución del mismo prompt, u otro modelo, puede producir código distinto. Se registró un único intento, que fue exitoso.
- **Representatividad de la variante humana:** se usó una implementación pública directa, no una optimizada por un experto; además se adaptó (bordes y normalización), lo que puede alterar levemente su rendimiento.
- **Misma familia de modelo en varias etapas:** la IA que generó la variante también asistió en la implementación de las optimizaciones aprobadas y en la redacción.
- **Variabilidad:** 78 de 256 configuraciones tienen un CV mayor que 10 %; no se aplicaron pruebas de significancia estadística.
- **Kernels binomiales:** la debilidad observada depende del rango de los pesos; otros kernels pueden favorecer o perjudicar a las variantes de manera distinta.

5. Conclusiones

La paralelización con OpenMP redujo el tiempo de la convolución en todas las variantes. En imágenes de 4096^2, el código generado directamente por IA alcanzó speedups de 36,9 a 61,8

frente a la versión secuencial con kernels de 3 a 7, frente a 5,8–6,7 de una implementación paralela escrita por personas. La mayor parte de esa ventaja proviene de optimizaciones de bajo nivel por núcleo, no de una mejor escalabilidad: con respecto a un solo hilo, las tres variantes escalaron de forma similar (6 a 10 veces con 16 hilos), con un beneficio limitado de HyperThreading para el código vectorizado. El código de IA fue correcto al primer intento en todas las pruebas, pero tuvo una debilidad dependiente de los datos (pesos mayores que 32 767) que redujo su speedup a 12,7 con kernels de 11×11; una optimización dirigida por el equipo y aplicada tras revisar el código la corrigió, con un speedup de 48,0. Para el desarrollo con IA, esto indica que la revisión del código generado y la medición sistemática siguen siendo necesarias, incluso cuando el código es correcto.

Declaración de uso de inteligencia artificial

La variante "IA directa" fue generada por Claude Opus 5.5 (Anthropic) a partir del prompt documentado en el repositorio. Las optimizaciones de la variante "IA + optimización dirigida por el equipo" fueron propuestas e implementadas con asistencia del mismo asistente, y su aplicación fue decidida por los autores. El asistente también se usó para preparar los scripts de medición y análisis y para apoyar la redacción del manuscrito; los autores revisaron el contenido y son responsables de él. La evaluación crítica del manuscrito se realizó con una herramienta de IA distinta, según se describe en la entrega.

Disponibilidad de código y datos

El código de las cuatro variantes, las pruebas de equivalencia, el registro de la interacción con la IA, los datos crudos (7 680 mediciones), el resumen estadístico y los scripts para reproducir el experimento (`scripts/run_bench.sh`) y este documento (`scripts/build_paper.sh`) están en el repositorio del proyecto: https://github.com/EstebanBautista/Midterm-2.

Referencias

[1] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, "Evaluation of OpenAI Codex for HPC parallel programming models kernel generation," in *Proc. 52nd Int. Conf. on Parallel Processing Workshops (ICPP-W '23)*. ACM, 2023, pp. 136–144.

[3] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?" in *Proc. 33rd Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC '24)*. Pisa, Italy: ACM, 2024.

[4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[5] Free Software Foundation, "Using the GNU compiler collection (GCC)," 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/

[6] Intel Corporation, “Intel intrinsics guide,” 2026. [Online]. Available: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/

[7] rdb987, “2DConvolution: 2D parallel convolution with MPI, OpenMP and hybrid strategy,” GitHub repository, commit d5b3922, 2016. [Online]. Available: https://github.com/rdb987/2DConvolution

[8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[9] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[10] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC ’15)*, 2015, pp. 73:1–73:12.

A. Resultados completos para imágenes de 4096^2

Tabla 3: Tiempo medio $\pm\sigma$ (30 repeticiones), speedup y eficiencia frente a la variante secuencial, imágenes de 4096^2.

Kernel	Variante	p	$\bar{T}$ (ms) $\pm\sigma$	S_p	E_p
3×3	Secuencial	1	162.56 ± 1.07	1.00	1.00
3×3	IA directa	1	34.31 ± 0.34	4.74	4.74
3×3	IA directa	2	17.42 ± 0.18	9.33	4.67
3×3	IA directa	4	9.45 ± 0.87	17.19	4.30
3×3	IA directa	8	5.08 ± 1.10	31.99	4.00
3×3	IA directa	16	4.41 ± 0.58	36.89	2.31
3×3	IA + opt. equipo	1	34.32 ± 0.22	4.74	4.74
3×3	IA + opt. equipo	2	17.57 ± 0.49	9.25	4.63
3×3	IA + opt. equipo	4	9.51 ± 0.89	17.10	4.28
3×3	IA + opt. equipo	8	5.09 ± 1.00	31.96	3.99
3×3	IA + opt. equipo	16	4.43 ± 0.54	36.66	2.29
3×3	Manual (GitHub)	1	254.08 ± 1.07	0.64	0.64
3×3	Manual (GitHub)	2	129.33 ± 2.96	1.26	0.63
3×3	Manual (GitHub)	4	65.97 ± 3.30	2.46	0.62
3×3	Manual (GitHub)	8	41.92 ± 2.89	3.88	0.48
3×3	Manual (GitHub)	16	26.36 ± 0.91	6.17	0.39
5×5	Secuencial	1	344.52 ± 2.26	1.00	1.00
5×5	IA directa	1	48.13 ± 0.42	7.16	7.16
5×5	IA directa	2	24.56 ± 1.75	14.03	7.01
5×5	IA directa	4	13.82 ± 1.92	24.93	6.23
5×5	IA directa	8	7.41 ± 1.54	46.47	5.81
5×5	IA directa	16	6.15 ± 0.72	56.01	3.50
5×5	IA + opt. equipo	1	48.30 ± 0.14	7.13	7.13
5×5	IA + opt. equipo	2	27.60 ± 6.05	12.48	6.24
5×5	IA + opt. equipo	4	14.73 ± 2.56	23.40	5.85
5×5	IA + opt. equipo	8	7.22 ± 1.61	47.74	5.97
5×5	IA + opt. equipo	16	6.04 ± 0.53	57.06	3.57
5×5	Manual (GitHub)	1	452.83 ± 2.10	0.76	0.76
5×5	Manual (GitHub)	2	233.02 ± 6.97	1.48	0.74
5×5	Manual (GitHub)	4	120.34 ± 6.58	2.86	0.72
5×5	Manual (GitHub)	8	70.46 ± 4.23	4.89	0.61
5×5	Manual (GitHub)	16	51.18 ± 0.92	6.73	0.42
7×7	Secuencial	1	597.69 ± 10.68	1.00	1.00
7×7	IA directa	1	67.51 ± 0.32	8.85	8.85
7×7	IA directa	2	34.67 ± 3.36	17.24	8.62
7×7	IA directa	4	17.95 ± 1.17	33.29	8.32
7×7	IA directa	8	9.68 ± 1.53	61.76	7.72
7×7	IA directa	16	10.77 ± 0.74	55.49	3.47
7×7	IA + opt. equipo	1	67.97 ± 1.96	8.79	8.79
7×7	IA + opt. equipo	2	34.35 ± 1.68	17.40	8.70
7×7	IA + opt. equipo	4	17.60 ± 0.73	33.97	8.49
7×7	IA + opt. equipo	8	10.24 ± 1.74	58.37	7.30
7×7	IA + opt. equipo	16	10.82 ± 0.67	55.22	3.45
7×7	Manual (GitHub)	1	745.45 ± 11.63	0.80	0.80
7×7	Manual (GitHub)	2	380.42 ± 8.82	1.57	0.79
7×7	Manual (GitHub)	4	201.13 ± 15.00	2.97	0.74
7×7	Manual (GitHub)	8	124.68 ± 15.97	4.79	0.60
7×7	Manual (GitHub)	16	88.62 ± 5.11	6.74	0.42
11×11	Secuencial	1	1429.85 ± 15.42	1.00	1.00
11×11	IA directa	1	710.73 ± 13.08	2.01	2.01
11×11	IA directa	2	366.31 ± 20.93	3.90	1.95
11×11	IA directa	4	189.69 ± 13.20	7.54	1.88
11×11	IA directa	8	121.57 ± 10.84	11.76	1.47
11×11	IA directa	16	112.55 ± 7.63	12.70	0.79
11×11	IA + opt. equipo	1	182.90 ± 1.19	7.82	7.82
11×11	IA + opt. equipo	2	94.42 ± 4.61	15.14	7.57
11×11	IA + opt. equipo	4	47.38 ± 2.03	30.18	7.55
11×11	IA + opt. equipo	8	36.81 ± 6.17	38.85	4.86
11×11	IA + opt. equipo	16	29.82 ± 1.46	47.95	3.00
11×11	Manual (GitHub)	1	1761.97 ± 18.12	0.81	0.81
11×11	Manual (GitHub)	2	909.69 ± 37.44	1.57	0.79
11×11	Manual (GitHub)	4	459.66 ± 14.62	3.11	0.78
11×11	Manual (GitHub)	8	333.69 ± 28.09	4.28	0.54
11×11	Manual (GitHub)	16	246.90 ± 2.03	5.79	0.36